React Authentication Module – Services, Hooks, Axios & Routing

1. Introduction

This document explains the implementation of a **React Authentication Module** using:

- Service-based architecture
- Axios for API communication
- React Hooks (`useState`, `useNavigate`)
- React Router
- `react-toastify` for notifications

The module includes **Login** and **Register** functionality with proper separation of concerns.

2. Backend Prerequisite: Enable CORS (Important)

Before integrating the **service layer in the frontend**, the backend must allow requests from the React application.

Since:

- Frontend runs on `http://localhost:5173`
- Backend runs on `http://localhost:3000`

The browser enforces **CORS (Cross-Origin Resource Sharing)** rules.

2.1 Install CORS Package

Run the following command in the backend project:

```
npm install cors
```

2.2 Configure CORS in `server.js`

To allow the React frontend to communicate with the backend APIs, CORS must be enabled in the backend server.

Code

```
const express = require('express')
const cors = require('cors')

const app = express()

app.use(cors())           // enable CORS
app.use(express.json())
```

Restart Backend Server

After enabling CORS, restart the backend server so the changes take effect.

3. Services Layer in React

What is a Service Layer?

A service layer is used to:

- Handle all backend API calls
- Separate business logic from UI components
- Improve reusability and maintainability

All API-related logic is placed inside the `services` folder.

3.1 config.js

Code

```
const config = {
    BASE_URL: 'http://localhost:3000'
}

export default config
```

Purpose

- Stores application-wide configuration values
- Centralizes backend base URL
- Prevents hardcoding URLs across files

3.2 userService.js

Code

```
import axios from 'axios'
import config from './config'

export async function loginUser(email, password) {
    const URL = config.BASE_URL + "/user/signin"
    const body = { email, password }
    const response = await axios.post(URL, body)
    return response.data
}

export async function registerUser(name, email, password, mobile) {
    const URL = config.BASE_URL + '/user/signup'
    const body = { name, email, password, mobile }
    const response = await axios.post(URL, body)
    return response.data
}
```

---

4. Axios

Axios is a **promise-based HTTP client** used in React applications to communicate with backend APIs.
It allows the frontend to send HTTP requests such as **GET, POST, PUT, and DELETE** to the server and handle responses efficiently.

4.1 Installing Axios

Before using Axios, it must be installed in the React project.

```
npm install axios
```

This command adds Axios as a dependency in the project.

4.2 Importing Axios

After installation, Axios must be imported into the file where API calls are required.

```
import axios from 'axios'
```

In this project, Axios is imported inside the service layer (`userService.js`) to keep API logic separate from UI components.

4.3 Using Axios for API Calls

Axios provides methods for different HTTP operations such as GET, POST, PUT, and DELETE.

Example: POST Request

```
const response = await axios.post(URL, body)
```

- **URL** → Backend API endpoint
- **body** → Data sent to the backend
- **response** → Server response object
- **response.data** → Actual response data returned by the backend

---

4.4 Advantages of Axios

- **Automatic JSON parsing**
  Converts request and response data automatically.
- **Cleaner syntax**
  Requires less boilerplate code compared to `fetch`.
- **Promise-based**
  Works smoothly with `async/await`.
- **Better error handling**
  Provides detailed error objects.
- **Supports interceptors**
  Useful for authentication tokens, logging, and request modification.

---

5. react-toastify

`react-toastify` is a popular React library used to display **non-blocking toast notifications** such as success messages, warnings, and errors.

These notifications improve user experience by providing instant feedback without interrupting the UI.

5.1 Installing react-toastify

Before using `react-toastify`, it must be installed in the React project.

```
npm install react-toastify
```

This command adds `react-toastify` as a dependency in the project.

5.2 Importing Toast Functions in Components

To display toast messages inside a component, import the `toast` function.

```
import { toast } from 'react-toastify'
```

The `toast` object provides different notification methods.

5.3 Toast Notification Types

Examples

```
toast.success('Operation successful')
toast.warn('Please enter valid input')
toast.error('Something went wrong')
toast.info('Information message')
```

Commonly Used Methods

- `toast.success()` → Displays success message
- `toast.warn()` → Displays warning message
- `toast.error()` → Displays error message
- `toast.info()` → Displays informational message

5.4 Importing ToastContainer in `App.jsx`

In addition to using `toast`, the application must render a **ToastContainer** component once, usually in `App.jsx`.

Import ToastContainer

```
import { ToastContainer } from 'react-toastify'
```

5.5 Adding ToastContainer to JSX

Add `<ToastContainer />` inside the main component JSX.

```
function App() {
  return (
    <>
      <ToastContainer />
      {/* Other components and routes */}
    </>
  )
}

export default App
```

The `ToastContainer` is responsible for rendering all toast notifications triggered anywhere in the application.

5.6 How react-toastify Works

```
User Action
   ↓
Validation / API Response
   ↓
toast.success() / toast.error()
   ↓
ToastContainer renders notification
```

5.7 Why ToastContainer Is Required

- Without `ToastContainer`, toast messages will not appear

- It should be added only once
- Recommended location is `App.jsx`

5.8 Advantages of react-toastify

- Non-blocking notifications
- Clean and professional UI
- Easy integration
- Supports auto-close and positioning
- Works across all components

---

6. React Hooks

React Hooks allow functional components to use React features such as **state** and **navigation** without writing class components.

6.1.1 useState()

`useState()` is a React Hook used to **manage state (data)** inside a functional component.

State represents data that can **change over time** and causes the component to **re-render** when updated.

6.1.2 Importing useState

Before using `useState`, it must be imported from React.

```
import { useState } from 'react'
```

6.1.3 Basic Syntax of useState

```
const [stateVariable, setStateFunction] = useState(initialValue)
```

Where:

- `stateVariable` → holds the current value of the state
- `setStateFunction` → function used to update the state

- `initialValue` → initial value of the state when the component loads

6.1.4 Example: Using useState for Name Field

```
const [name, setName] = useState('')
```

Explanation:

- `name` stores the current value of the input
- `setName()` updates the state
- `''` is the initial value

6.1.5 How useState Works (Step-by-Step)

1. Component loads with initial state
2. User types into input field
3. `onChange` event is triggered
4. `setState()` updates the state
5. React re-renders the component

6.1.6 useState with Input Field

```
<input
  type="text"
  value={name}
  onChange={e => setName(e.target.value)}
/>
```

Flow:

```
User Input
   ↓
onChange Event
   ↓
setName()
   ↓
State Update
   ↓
Component Re-render
```

6.1.7 Why useState is Required

❌ Incorrect (React will not track changes):

```
let name = ''
```

✔ Correct:

```
const [name, setName] = useState('')
```

6.1.8 Multiple useState Hooks

```
const [name, setName] = useState('')
const [email, setEmail] = useState('')
const [password, setPassword] = useState('')
const [mobile, setMobile] = useState('')
```

Each state variable is independent.

6.1.9 Using State Values

State values are commonly used for:

- Form validation
- API calls
- Conditional rendering

Example:

```
registerUser(name, email, password, mobile)
```

6.1.10 Controlled Components

When inputs are controlled by state:

```
<input value={name} onChange={e => setName(e.target.value)} />
```

React controls the input value.

6.1.11 Important Rules

- Hooks must be called at the top level
- Hooks cannot be called conditionally
- State resets on page refresh

6.2.1 useNavigate()

`useNavigate()` is a React Router hook used for **programmatic (dynamic) navigation** in React applications.

It allows navigation to different routes **based on logic**, such as after login, registration, or form submission.

6.2.2 Importing useNavigate

Before using `useNavigate`, it must be imported from `react-router-dom`.

```
import { useNavigate } from 'react-router-dom'
```

6.2.3 Basic Syntax

```
const navigate = useNavigate()
```

- `navigate` is a function returned by the hook
- It is used to change routes dynamically

6.2.4 Example Usage

```
const navigate = useNavigate()
navigate('/home')
```

This redirects the user to the `/home` route.

6.2.5 Why useNavigate is Needed

In React applications:

- `<Link>` is used for **static navigation**
- `useNavigate()` is used for **dynamic navigation**

Common use cases:

- Redirect after successful login
- Redirect after registration
- Redirect after form submission
- Conditional navigation

6.2.6 useNavigate in Login Flow (Example)

```
const signin = async () => {
  const result = await loginUser(email, password)

  if (result.status === 'success') {
    navigate('/home')
  }
}
```

Navigation occurs only if login is successful.

6.2.7 useNavigate in Register Flow (Example)

```
if (result.status === 'success') {
  navigate('/')
}
```

Redirects user to Login page after successful registration.

6.2.8 How useNavigate Works (Flow)

```
User Action
   ↓
Condition / Logic
   ↓
navigate('/route')
   ↓
URL Changes
   ↓
React Router Renders New Component
```

6.2.9 Navigation Options

Navigate Back

```
navigate(-1)
```

Navigate Forward

```
navigate(1)
```

6.2.10 Replace History Entry

```
navigate('/home', { replace: true })
```

Prevents user from navigating back to the previous page.

6.2.11 useNavigate vs Link

useNavigate	Link
Programmatic navigation	Static navigation
Used inside logic	Used in JSX
Conditional	Always visible

useNavigate	Link
Ideal for redirects	Ideal for menus

6.2.12 Rules for Using useNavigate

- Must be used inside a React component
- Component must be wrapped in `<BrowserRouter>`
- Cannot be used outside React Router context

---

7. Register Component Flow

1. User enters details
2. Validation is performed
3. API call is made using service
4. Toast message is shown
5. User is redirected

---

8. Login Component Flow

1. User enters credentials
2. Validation is performed
3. Authentication API is called
4. Toast notification is displayed
5. User navigates to Home page

---

9. Architecture Flow

```
UI Component
   ↓
useState
   ↓
Service Layer (Axios)
   ↓
Backend API
   ↓
Toast
   ↓
useNavigate
```
